



SENAI – Serviço Nacional
de Aprendizagem
Industrial
CFP- Cândido Athayde
(NFP-Raimundo de Sousa Lima)
Atividade

3º Ano (A)
Comp. Curricular:
Professor(a): Cleiton
Data: ____/____/____

Aluno (a): Pedro Paulo Lima Sousa

Nº: 25

Questão 1 - Ao criar um algoritmo para processar uma lista de 50 produtos, como um desenvolvedor pode automatizar o processo de leitura e exibição de cada nome de produto usando uma linguagem de programação?

- (A) Escrevendo uma instrução de leitura e exibição individual para cada um dos 50 produtos.
- (B) Definindo uma nova variável para cada produto, sem usar uma estrutura de dados.
- (C) Utilizando apenas uma função que lê e exibe o primeiro produto da lista.
- (D) Criando um algoritmo que exige intervenção manual para cada produto processado.
- (E) **Empregando uma estrutura de repetição (loop) para iterar sobre a lista de produtos.**

Questão 2 - Em programação, a modularização é uma prática essencial para o desenvolvimento de sistemas complexos. Qual o principal benefício da modularização de um programa em funções ou procedimentos distintos?

- (A) Aumenta a velocidade de execução do programa, pois as funções são compiladas em paralelo.
- (B) Garante que o programa nunca apresente erros, eliminando a necessidade de testes.
- (C) Reduz a quantidade total de memória RAM necessária para a execução do software.
- (D) Impede que qualquer parte do código seja alterada por outros desenvolvedores, protegendo a propriedade intelectual.
- (E) **Facilita a manutenção, depuração e reuso do código, além de promover a organização e legibilidade.**

Questão 3- No contexto das redes de computadores, qual é a principal função do protocolo TCP (Transmission Control Protocol) na camada de transporte do modelo OSI/TCP-IP?

- (A) Realizar o roteamento de pacotes entre diferentes redes IP, encontrando o melhor caminho.
- (B) Atribuir endereços IP dinamicamente aos dispositivos em uma rede local.
- (C) Converter nomes de domínio em endereços IP para acesso a recursos na internet.
- (D) Criptografar todo o tráfego de dados para garantir a segurança e privacidade das comunicações.
- (E) **Garantir a entrega confiável e ordenada dos dados entre as aplicações, com controle de fluxo e retransmissão de erros.**

Questão 4- A arquitetura de computadores é fundamental para o desempenho de sistemas digitais. Explique a função principal da Unidade Lógica e Aritmética (ULA) e da Unidade de Controle (UC) em um processador. Como a interação entre essas duas unidades é crucial para a execução de instruções de um programa?

Porque enquanto uma realiza os cálculos que aparecem a outra organiza de forma controlada, ajudando assim, no funcionamento do programa.

Questão 5. - As estruturas de repetição, ou laços (loops), são vitais para automatizar tarefas repetitivas em programação. Explique a diferença fundamental entre um laço `for` e um laço `while`. Em que tipo de situação você escolheria usar um `for` em vez de um `while`, e vice-versa? Apresente um breve exemplo para cada tipo de laço.

O laço `for` serve para determinar quantas vezes algo vai ser repetido. O laço `while` é usado quando a repetição depende de uma condição.

Eu usaria o `for` em uma situação em que eu deveria empilhar algo, por exemplo, eu teria 5 sacos de arroz, e eu deveria colocar em 5 lugares diferentes, eu repetiria a ação 5 vezes.

Eu usaria o `while` em uma situação que eu deveria encher o pneu até chegar em determinada pressão.

Questão 6. - Qual dos seguintes conceitos em programação descreve uma função que é passada como argumento para outra função, para ser executada em um momento posterior?

- (A) Função aninhada
- (B) Função recursiva
- (C) Método estático
- (D) Função polimórfica
- (E) **Callback (retorno de chamada)**

Questão 7- Uma função recursiva é definida para calcular o fatorial de um número inteiro positivo n . Se a função é chamada com $fatorial(4)$, quantas chamadas à função $fatorial$ (incluindo a chamada inicial) ocorrerão até que o resultado final seja obtido?

- (A) 1
- (B) 3
- (C) 5
- (D) 8
- (E) 4

Questão 8- Em linguagens de programação que suportam programação assíncrona, qual a principal vantagem do uso de funções de 'callback' em comparação com abordagens síncronas, como a espera bloqueante?

- (A) Reduzir a complexidade do código, tornando-o mais fácil de ler e manter.
- (B) Permitir a execução de operações assíncronas sem bloquear o fluxo principal do programa, melhorando a responsividade e o paralelismo.
- (C) Garantir a execução sequencial estrita de todas as operações em um sistema.
- (D) Diminuir o consumo de memória RAM ao evitar a criação de novas threads para cada operação.
- (E) Eliminar a necessidade de tratamento de erros em operações de entrada e saída.

Questão 9- Explique o conceito de 'callback hell' (inferno de callbacks) e discuta as suas principais desvantagens no desenvolvimento de software. Apresente duas abordagens comuns para mitigar esse problema em linguagens de programação modernas.

Callback hell é quando o código fica com muitos callbacks dentro de callbacks, deixando tudo confuso e difícil de entender e manter. Isso atrapalha a leitura, dificulta a correção de erros e pode tornar o programa desorganizado. Para diminuir esse problema, uma forma é separar cada etapa em funções menores, deixando o código mais organizado. Outra forma é usar uma escrita mais moderna e simples, que evita muitos blocos aninhados e deixa o fluxo do programa mais fácil de acompanhar.

Questão 10- Considere o uso de 'callbacks' em um cenário de tratamento de eventos em uma interface gráfica de usuário (GUI). Descreva como os 'callbacks' são empregados para responder às interações do usuário (como um clique de botão) e explique a importância desse modelo para a reatividade da aplicação. Qual o impacto da ordem de registro de múltiplos 'callbacks' para o mesmo evento?

Callbacks são funções que o programa executa quando o usuário faz algo, como clicar em um botão. Eles deixam a aplicação mais rápida e responsiva, porque ela reage aos eventos em vez de ficar parada esperando. Quando há vários callbacks para o mesmo evento, a ordem de registro pode influenciar a ordem de execução, dependendo da linguagem ou do framework usado.